



**Facultad de
Ciencias**

UNAM

PRÁCTICA 7 - SUMA PARALELA DE VECTORES

ALGORITMOS PARALELOS

Baltodano Cuevas Erick Joel

Profesor: Mario Arturo Nieto Butron

1. Objetivos

El objetivo de esta práctica fue implementar un programa en lenguaje C que realizara operaciones de suma binaria utilizando programación paralela mediante OpenMP. Además, se buscó analizar el comportamiento del programa al ejecutar las operaciones con distintos números de hilos, observando el tiempo de ejecución en las etapas de conversión y suma.

2. Metodología

3. Metodología

Para el desarrollo de la práctica se utilizó el archivo `Suma.c`, el cual implementa una suma binaria paralela entre dos números enteros no negativos.

La metodología seguida fue la siguiente:

1. Se recibieron como parámetros de entrada:
 - El número de hilos a utilizar.
 - Dos números enteros en decimal.
2. Cada número decimal fue convertido a representación binaria mediante la función `decimal_a_binario()`.
3. Se utilizó OpenMP para paralelizar distintas secciones del programa:
 - Conversión simultánea de ambos números a binario.
 - Cálculo paralelo de operaciones XOR y AND para la suma binaria.
 - Conversión del resultado binario nuevamente a decimal utilizando reducción paralela.
4. La suma binaria se realizó considerando:
 - Generación de carries.
 - Propagación de carries.
 - Construcción del resultado final.
5. Finalmente, el programa mostró:
 - Los números en decimal y binario.
 - El resultado de la suma.
 - El tiempo de ejecución de cada etapa.

4. Resultados

- $88 + 42$

```
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=4 Numero1=88 Numero2=42
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 4 88 42
|
|      SUMA BINARIA PARALELA CON HILOS (OpenMP)      |
|-----|
| Número de hilos      : 4                             |
|-----|
| Número 1 (dec)       : 88                             |
| Número 1 (bin)       : 1011000                       |
|-----|
| Número 2 (dec)       : 42                             |
| Número 2 (bin)       : 101010                       |
|-----|
| Resultado (bin)      : 10000010                      |
| Resultado (dec)      : 130                           |
|-----|
|----- TIEMPOS -----|
| Conversión dec-bin   : 1836.3690 µs                   |
| Suma binaria        : 1789.2940 µs                   |
|-----|
| TOTAL               : 3625.6630 µs                   |
|-----|
rm -r src/Suma
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$
```

- $342 + 58$

```
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=4 Numero1=342 Numero2=58
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 4 342 58
|
|      SUMA BINARIA PARALELA CON HILOS (OpenMP)      |
|-----|
| Número de hilos      : 4                             |
|-----|
| Número 1 (dec)       : 342                             |
| Número 1 (bin)       : 101010110                     |
|-----|
| Número 2 (dec)       : 58                             |
| Número 2 (bin)       : 111010                       |
|-----|
| Resultado (bin)      : 110010000                     |
| Resultado (dec)      : 400                           |
|-----|
|----- TIEMPOS -----|
| Conversión dec-bin   : 1070.6800 µs                   |
| Suma binaria        : 3316.3770 µs                   |
|-----|
| TOTAL               : 4387.0570 µs                   |
|-----|
rm -r src/Suma
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$
```

- $550 + 242$

```

baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=4 Numero1=550 Numero2=242
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 4 550 242
SUMA BINARIA PARALELA CON HILOS (OpenMP) ||
-----
Número de hilos : 4
-----
Número 1 (dec) : 550
Número 1 (bin) : 1000100110
-----
Número 2 (dec) : 242
Número 2 (bin) : 11110010
-----
Resultado (bin) : 1100011000
Resultado (dec) : 792
-----
TIEMPOS -----
Conversión dec-bin : 3781.7430 µs
Suma binaria : 387.1610 µs
-----
TOTAL : 4168.9040 µs
rm -r src/Suma
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$

```

- $43 + 22$

```

baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=4 Numero1=43 Numero2=22
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 4 43 22
SUMA BINARIA PARALELA CON HILOS (OpenMP) ||
-----
Número de hilos : 4
-----
Número 1 (dec) : 43
Número 1 (bin) : 101011
-----
Número 2 (dec) : 22
Número 2 (bin) : 10110
-----
Resultado (bin) : 1000001
Resultado (dec) : 65
-----
TIEMPOS -----
Conversión dec-bin : 774.0390 µs
Suma binaria : 2487.3150 µs
-----
TOTAL : 3261.3540 µs

```

- $682 + 750$

```

baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=4 Numero1=682 Numero2=750
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 4 682 750
SUMA BINARIA PARALELA CON HILOS (OpenMP) ||
-----
Número de hilos : 4
-----
Número 1 (dec) : 682
Número 1 (bin) : 1010101010
-----
Número 2 (dec) : 750
Número 2 (bin) : 1011101110
-----
Resultado (bin) : 10110011000
Resultado (dec) : 1432
-----
TIEMPOS -----
Conversión dec-bin : 2042.3370 µs
Suma binaria : 2436.9660 µs
-----
TOTAL : 4479.3030 µs
rm -r src/Suma
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$

```

- Suma de 15 cifras

```

baltimus@Dev:~/github/Practicas_Paralelos/Practica7$ make suma N=7 Numero1=111111111111111111 Numero2=2222222222222222
gcc -fopenmp -o src/Suma src/Suma.c -lm
./src/Suma 7 1111111111111111 2222222222222222
SUMA BINARIA PARALELA CON HILOS (OpenMP)
-----
Número de hilos : 7
-----
Número 1 (dec) : 1111111111111111
Número 1 (bin) : 11001010000111000010010010011101111000111000111
-----
Número 2 (dec) : 2222222222222222
Número 2 (bin) : 110010100001110000100100100111011110001110001110
-----
Resultado (bin) : 10010111100101010001101101110110011010101010101
Resultado (dec) : 3333333333333333
-----
TIEMPOS -----
Conversión dec-bin : 1296.9770 µs
Suma binaria : 1162.8020 µs
-----
TOTAL : 2459.7790 µs
rm -r src/Suma
baltimus@Dev:~/github/Practicas_Paralelos/Practica7$

```

5. Conclusiones

A partir de esta práctica se concluye que OpenMP facilita la implementación de programación paralela en lenguaje C mediante directivas simples.

El uso de múltiples hilos permitió dividir ciertas operaciones del programa, especialmente en la conversión y procesamiento de bits, reduciendo el tiempo de ejecución en comparación con una implementación completamente secuencial.

También se reforzaron conceptos importantes como:

- Representación binaria de números.
- Operaciones lógicas XOR y AND.
- Manejo de carries en sumas binarias.
- Paralelismo con OpenMP.
- Medición de tiempos de ejecución.

En general, la práctica permitió comprender cómo aplicar técnicas de programación concurrente para mejorar el rendimiento de algoritmos computacionales.